

Краткий список серьезных недоработок дипломного проекта

Проект: "Интерфейс управления жизненным циклом рабочего места программиста". Основа проекта рабочая, но ниже перечислены замечания, которые сильнее всего влияют на защищаемость и качество MVP.

1. Критично для демонстрации

Проблема	Суть замечания
Ключевые действия не выведены во frontend	На backend есть API для создания/изменения/удаления артефактов, связей и кода, но интерфейс показывает не все эти сценарии. Например, кнопка создания артефакта есть, но полноценного пользовательского сценария создания не видно.
Пользователь не может полноценно собрать модель через UI	Система хорошо показывает демо-проект, граф и анализ влияния, но хуже демонстрирует ручное создание артефактов и связей. Для темы диплома это слабое место: интерфейс должен управлять связанными артефактами, а не только показывать заранее подготовленные данные.
Нет полноценного управления связями в интерфейсе	Связи являются центральной частью онтологической модели. Нужно хотя бы минимальное окно: исходный артефакт, тип связи, целевой артефакт, комментарий, создание/удаление связи.
Редактор/просмотрщик кода не реализован в UI	Зависимость для Monaco Editor есть, а сценарий работы с фрагментами кода в интерфейсе не раскрыт. По ТЗ код должен быть одним из артефактов жизненного цикла.

2. Серьезные backend-недоработки

Проблема	Что исправить
Не проверяется уникальность кода артефакта внутри проекта	По ТЗ код артефакта должен быть уникален в рамках project_id. Иначе возможны две записи REQ-001 или MOD-001 в одном проекте.
Не проверяется существование type_id при создании артефакта	Сейчас можно передать несуществующий тип артефакта, если проверка не добавлена на уровне API/сервиса.
Не проверяется существование relation_type_id при создании связи	Связь может ссылаться на несуществующий тип связи. Это ломает целостность онтологической модели.
Удаление артефакта может оставить битые связи	Перед удалением артефакта нужно удалять входящие и исходящие ArtifactRelation или запрещать удаление до очистки связей.
Не хватает защиты от дублей связей	Желательно запретить повторную одинаковую связь: source, target, relation_type. Иначе граф может засоряться одинаковыми ребрами.
metadata_json задан через default={}	Лучше заменить на default=dict. Это стандартная правка, чтобы избежать проблем с изменяемым значением по умолчанию.
Тесты не изолированы от рабочей БД	Тест impact-сценария лучше запускать на отдельной временной SQLite-БД, а не через обычную конфигурацию приложения.

3. Серьезные frontend-недоработки

Проблема	Что исправить
Нет полноценного CRUD через интерфейс	Нужны минимум: создание артефакта, редактирование артефакта, удаление артефакта с подтверждением.

Проблема	Что исправить
Нет отдельной карточки артефакта	Пользователю полезно видеть тип, код, описание, статус, приоритет, метаданные, входящие/исходящие связи и связанный код.
Нет удобного создания проекта через UI	Если создание проекта уже есть в API, стоит вывести простую форму на frontend.
Слабая обработка ошибок	Нужно показывать понятные сообщения при ошибках API: не удалось загрузить данные, дублирующийся код, недопустимая связь и т.д.
API URL захардкожен	Вместо <code>http://localhost:8000</code> лучше использовать <code>VITE_API_URL</code> и <code>.env.example</code> .

4. Репозиторий, зависимости и документация

Проблема	Что сделать
В репозитории есть <code>__pycache__</code> и <code>.рус</code>	Удалить из Git и добавить в <code>.gitignore</code> : <code>**/__pycache__/</code> и <code>*.py[cod]</code> .
<code>requirements.txt</code> без закреплённых версий	Для воспроизводимости лучше зафиксировать версии зависимостей после проверки запуска.
CORS открыт полностью	Для локального MVP допустимо, но в пояснении указать как демонстрационное ограничение. Лучше ограничить <code>origins</code> для frontend.
Формулировка README может путать тему	Держать единообразно: "рабочее место программиста", а не просто "рабочие места".
Не хватает <code>.env.example</code>	Добавить пример переменных окружения без секретов.

5. Минимальный план исправлений перед защитой

1. Сделать форму создания артефакта во frontend.
2. Сделать форму создания и удаления связи между артефактами.
3. Добавить простой экран/модалку для просмотра и сохранения фрагмента кода.
4. Добавить проверки backend: уникальный код артефакта, существование `type_id` и `relation_type_id`, удаление связанных отношений.
5. Почистить репозиторий от `__pycache__` и `.рус`.
6. Добавить `.env.example`, закрепить зависимости и поправить README под формулировку темы.

Итог: проект уже выглядит как рабочий дипломный MVP, а не как пустая заготовка. Главный риск - интерфейс пока не полностью раскрывает управление артефактами, связями и кодом. Исправлять нужно точно, без переписывания архитектуры.